MASTER · MANAGEMENT DE L'IA EN SANTÉ · FIN DE COURS

Guide des algorithmes

Ce que fait chaque méthode vue en cours, à quoi elle sert, quand la choisir et comment éviter ses pièges. À garder sous la main pour vos projets.

Auteur : Mamadou Alpha Hawa BALDE

Bloc 1 · Data science

SÉANCES 1 À 3

- Le workflow d'un projet
- Qualité des données
- Le split d'abord
- Features et Pipeline
- Data leakage

Bloc 2 · Machine learning

SÉANCES 4 À 7

- Baselines
- Linear regression, Ridge, Lasso
- Logistic regression, KNN, decision tree
- Métriques et seuil
- Cross-validation et tuning
- Random forest, gradient boosting
- Interprétation

Bloc 3 · Deep learning

SÉANCES 8 À 11

- Neurone, MLP, backpropagation
- Entraîner avec PyTorch
- Overfitting et régularisation
- Embeddings, texte, fusion

Hors séances · Non supervisé

NOTEBOOKS 90 ET 91

- k-means, hiérarchique, DBSCAN
- PCA
- Anomaly detection

Sommaire

Le workflow et les règles d'or	2
Choisir un algorithme	3
Régression	4
Classification	5
Ensembles et interprétation	6
Évaluer : métriques et seuil	7
Valider et régler	8
Deep learning : du neurone au réseau	9
Deep learning : entraîner et généraliser	10
Deep learning : représentations et texte	11
Non supervisé	12
Synthèse, pièges et checklist	13

Comment lire une fiche

Interprétabilité ●●●●○ peut-on expliquer une prédiction à un clinicien ?

Non-linéarités ●●●●○ sait-il suivre des courbes et des interactions sans aide ?

Vitesse ●●●●○ entraînement et prédiction sur un ordinateur portable.

Scaling : requis faut-il standardiser les features (StandardScaler) ?

QUAND L'UTILISER LIMITES puis les réglages clés et le piège classique.

Les cinq réflexes, quel que soit l'algorithme

1. **Baseline** — un modèle naïf fixe le plancher à battre
2. **Split d'abord** — le test set reste scellé jusqu'à la fin
3. **Pipeline** — tout preprocessing appris sur le train set seulement
4. **Métrique** — choisie à partir du coût des erreurs
5. **Incertitude** — une différence plus petite que le bruit n'en est pas une

Trois questions à poser devant tout score

Comparé à quoi ?
à une baseline, sur les mêmes données et les mêmes folds

Mesuré comment ?
quel split, quelle validation, quelle métrique, quel seuil

Avec quelle incertitude ?
l'écart entre folds ou entre seeds : une différence plus petite n'existe pas

Crédits et droits

- **Auteur** : Mamadou Alpha Hawa BALDE. Conception, rédaction, schémas et graphiques.
- **Graphiques** : créations originales, produites en Python (matplotlib, scikit-learn) sur données synthétiques. Ils montrent un mécanisme, pas un résultat du projet.
- **Droits** : © 2026 Mamadou Alpha Hawa BALDE. Tous droits réservés. Toute reproduction ou diffusion doit mentionner l'auteur.

Le workflow et les règles d'or

Avant tout algorithme : une question claire, des données vérifiées, un split honnête et un Pipeline. La plupart des erreurs graves viennent d'ici, pas du choix du modèle.

Les dix étapes d'un projet

1. **Question** — quelle décision ? quelle target ? quel coût d'erreur ?
2. **Données** — qualité, manquants, doublons, unités
3. **Split** — train, validation, test avant toute exploration chiffrée
4. **Exploration** — distributions, liens, valeurs aberrantes (EDA)
5. **Features** — nettoyer, encoder, créer, dans un Pipeline
6. **Baseline** — Dummy, puis modèle simple
7. **Modèles** — du plus simple au plus riche
8. **Validation** — cross-validation adaptée aux groupes
9. **Interprétation** — importance, erreurs par sous-groupe
10. **Communication** — chiffres traduits pour le client, limites écrites

Le split d'abord



Proportions usuelles : 60 à 70 %, 15 à 20 %, 15 à 20 %.

- On sépare **avant** de regarder les chiffres : sinon nos choix s'inspirent du test set et le score final devient optimiste.
- Le **test set** sert une seule fois, à la fin. Pour choisir entre modèles : validation ou cross-validation.
- Plusieurs lignes par patient (ou par hôpital) ? On sépare par **groupe** (GroupShuffleSplit, GroupKFold).
- Données datées ? On entraîne sur le passé, on teste sur le futur.

Le Pipeline

Données brutes → SimpleImputer (manquants) → OneHotEncoder ou StandardScaler (ColumnTransformer) → Modèle

```
pipe = Pipeline([("prep", preprocess),
                 ("model", LogisticRegression())])
pipe.fit(X_train, y_train)  # apprend tout sur le train
pipe.predict(X_test)        # applique sans réapprendre
```

Tout ce qui s'apprend sur des données (moyenne d'imputation, échelle du scaler, catégories de l'encoder) s'apprend sur le train set seulement. Le Pipeline le garantit, y compris dans la cross-validation.

Data leakage : quatre formes

Fuite	Exemple	Remède
Preprocessing appris sur tout	scaler ou imputation avant le split	Pipeline
Feature qui contient la réponse	« date de sortie » pour prédire la durée de séjour	se demander : la connaîtrai-je au moment de prédire ?
Même individu des deux côtés	un patient en train et en test	split par groupe
Le futur dans le passé	moyenne calculée sur toute la période	split temporel

Un score trop beau est un signal d'alerte : chercher la fuite avant de fêter.

Préparer chaque type de feature

Type	Préparation habituelle
Numérique	imputation (médiane), puis StandardScaler pour les modèles à distance ou à gradient
Catégorielle	OneHotEncoder(handle_unknown="ignore") ; embeddings si des centaines de modalités
Date	extraire mois, jour de semaine, ancienneté
Texte libre	TF-IDF, ou embeddings d'un encoder pré-entraîné
Target très asymétrique	modéliser log1p(y), puis retraduire l'erreur en unités réelles
Valeurs manquantes	se demander pourquoi elles manquent : l'absence peut être une information

Contrôler la qualité des données

- [ ] valeurs manquantes : combien, où, et pourquoi ?
- [ ] doublons : mêmes patients, mêmes séjours saisis deux fois ;
- [ ] valeurs impossibles (âge négatif, tension à 0) et unités mélangées ;
- [ ] déséquilibre des classes : quel taux de positifs ?
- [ ] dérive dans le temps : les données récentes ressemblent-elles aux anciennes ?

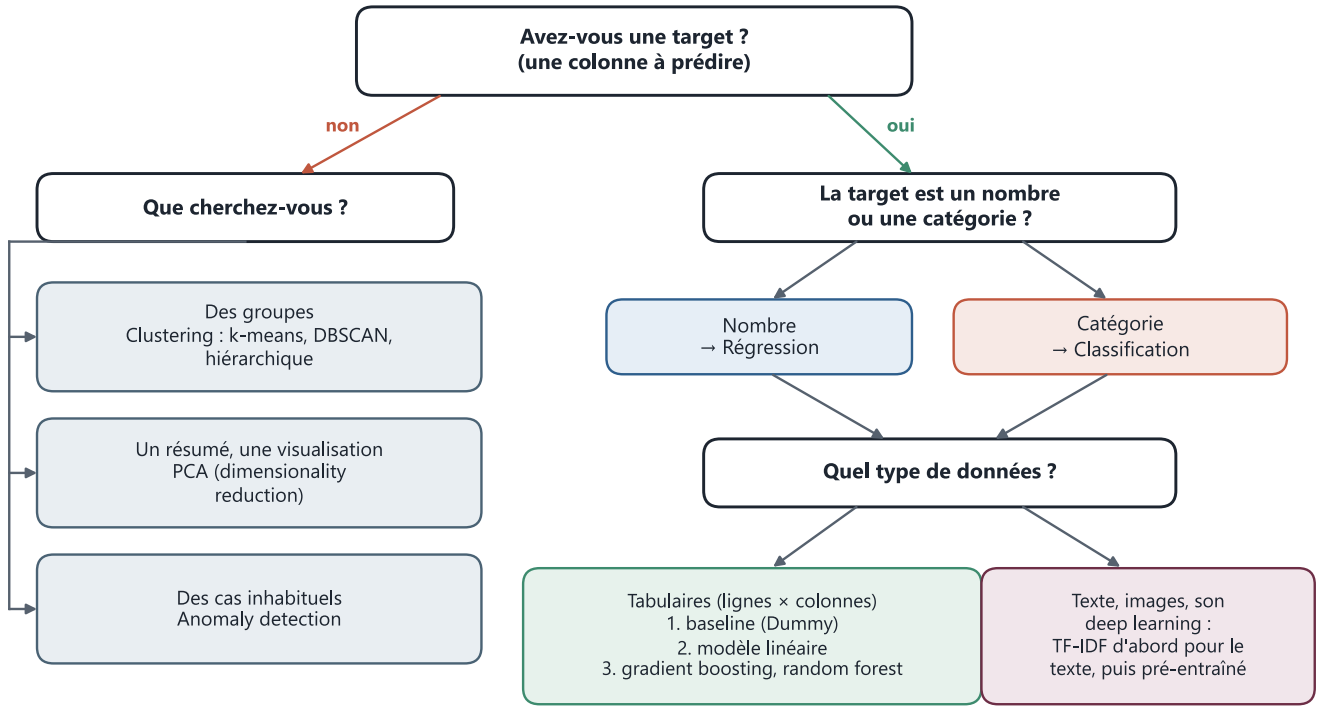
Quel graphique pour quelle question ?

Question	Graphique
Comment se répartit une variable ?	histogramme, boxplot
Deux variables numériques sont-elles liées ?	scatter plot
Une mesure diffère-t-elle entre groupes ?	boxplot par groupe
Comment évolue-t-elle dans le temps ?	courbe
Quelles features vont ensemble ?	heatmap des corrélations
Quelles proportions ?	barres (pas de camembert à dix parts)

MÉTHODE

Choisir un algorithme

Trois questions suffisent pour démarrer : y a-t-il une target, de quel type, et sur quelles données ? Ensuite, on compare honnêtement, du plus simple au plus riche.

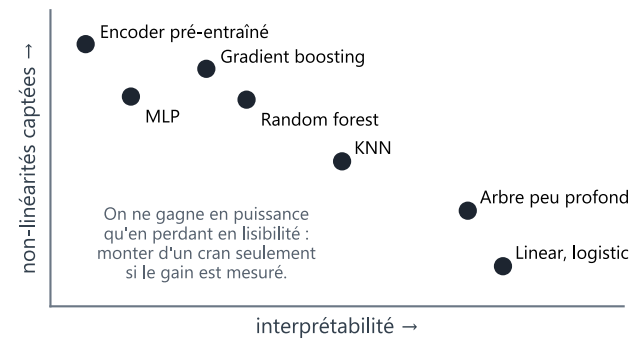


Repères rapides

Situation	Premier choix	Pourquoi
Toujours, en premier	DummyRegressor ou DummyClassifier	fixe le plancher : un modèle qui ne le bat pas n'apprend rien d'utile
Peu de lignes (quelques centaines)	modèle linéaire régularisé, arbre peu profond	moins de variance, résultats plus stables
Tabulaire, milliers de lignes	gradient boosting (HistGradientBoosting), random forest	non-linéarités et interactions, sans scaling
Expliquer chaque décision	logistic regression, arbre de profondeur 3	coefficients et règles lisibles par un clinicien
Classes très déséquilibrées	n'importe lequel + bonne métrique + seuil choisi	le modèle ne règle pas le déséquilibre, la métrique et le seuil oui
Texte libre (comptes rendus)	TF-IDF + logistic regression, puis embeddings	baseline rapide, forte et lisible
Images, signaux	réseau pré-entraîné (transfer learning)	on n'entraîne pas de zéro avec peu de données
Pas de target	clustering, PCA	explorer, segmenter, résumer

Interprétabilité contre puissance

Le compromis, en ordre de grandeur



Positions indicatives : elles varient avec les réglages (un arbre profond devient vite illisible).

Simple d'abord

On ajoute de la complexité seulement si elle **gagne nettement** en validation, au-delà de l'écart entre folds. À score égal, on garde le modèle le plus simple : plus facile à expliquer, à maintenir, à auditer.

Ce qui compte plus que l'algorithme

- la qualité des features et l'absence de data leakage ;
- une validation qui respecte la structure des données ;
- une métrique et un seuil qui reflètent le coût réel des erreurs.

Régression : prédire un nombre

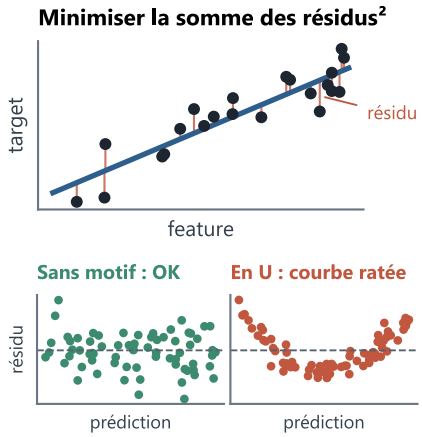
Durée de séjour, coût d'un traitement, prix. On compare toujours à une baseline, et on traduit l'erreur dans l'unité du client.

Baseline : DummyRegressor. Prédit toujours la moyenne (ou la médiane) du train set : $R^2 \approx 0$ par construction. C'est le plancher. Un modèle qui ne le bat pas n'a rien appris d'utile.

Linear regression

`LinearRegression()`

Interprétabilité ●●●●● Non-linéarités ●○○○○ Vitesse ●●●●● Scaling : inutile (sauf régularisé)



L'idée. Trouve la droite (l'hyperplan) qui minimise la somme des carrés des résidus (OLS). Chaque coefficient dit de combien la prédiction varie quand une feature augmente d'une unité, les autres restant fixes.

QUAND L'UTILISER

- relation à peu près linéaire ;
- besoin d'expliquer les effets ;
- peu de données ; baseline sérieuse.

LIMITES

- rate les courbes et interactions, sauf si on les crée ;
- sensible aux valeurs aberrantes ;
- coefficients instables si les features sont corrélées.

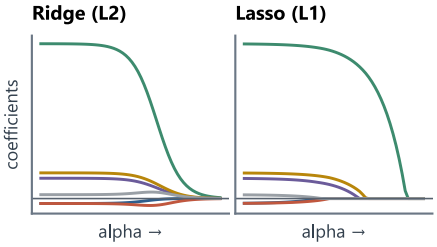
Réglages clés. aucun. On diagnostique avec le **graphique des résidus** : un motif (courbe, entonnoir) signale une relation manquée.

Piège. un coefficient n'est pas un effet causal. Il dépend des autres features présentes : ajouter une feature corrélée peut le faire changer de signe.

Ridge et Lasso

`Ridge(alpha=1.0)` · `Lasso(alpha=0.01)`

Interprétabilité ●●●●○ Non-linéarités ●○○○○ Vitesse ●●●●● Scaling : requis



L'idée. Une régression linéaire plus une pénalité sur la taille des coefficients (regularization). **Ridge** (L2) les rétrécit tous ; **Lasso** (L1) en met certains exactement à zéro : il sélectionne des features.

QUAND L'UTILISER

- beaucoup de features, ou features corrélées ;
- signes d'overfitting (train bien meilleur que validation) ;
- Lasso : envie d'un modèle plus court.

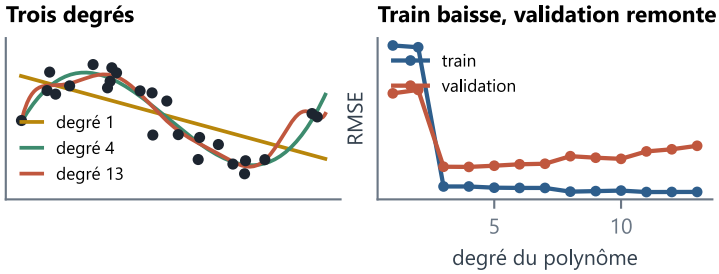
LIMITES

- inutile s'il n'y a pas d'overfitting ;
- Lasso peut écarter une feature utile parmi deux corrélées ;
- scaling obligatoire, sinon la pénalité est injuste.

Réglages clés. alpha : plus il est grand, plus le modèle est simple. On le choisit par cross-validation, jamais sur le test set.

Polynomial features et bias-variance

`PolynomialFeatures(degree=3)`



L'idée. Ajouter x^2, x^3 ou des produits de features permet à un modèle linéaire de suivre une courbe. Plus le degré monte, plus l'erreur train baisse ; l'erreur de validation baisse puis remonte : c'est l'**overfitting**.

QUAND L'UTILISER

- une relation courbe connue (âge et risque, dose et effet) ;
- illustrer le compromis bias-variance.

LIMITES

- le nombre de features explose avec le degré ;
- extrapolation dangereuse hors de la plage observée.

Réglages clés. degree, choisi par validation curve. Souvent combiné à Ridge.

Mesurer l'erreur d'une régression

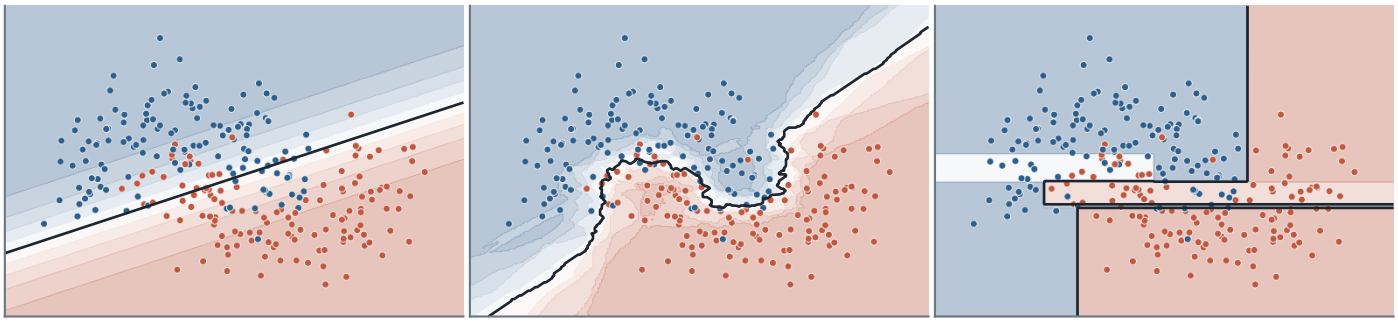
Métrique	Ce qu'elle dit	À préférer quand
MAE	erreur moyenne, dans l'unité de la target	on l'explique au client ; valeurs extrêmes
RMSE	pénalise davantage les grosses erreurs	une grosse erreur coûte plus que plusieurs petites
R^2	part de la variance expliquée ; 0 = baseline, peut être négatif	on compare à la baseline

Vu dans le cours. Ridge n'a presque rien changé par rapport à OLS : sans overfitting, il n'y a rien à corriger. L'erreur de validation du polynôme s'est effondrée **brutalement** à partir d'un certain degré, pas progressivement. Et une RMSE en log se retraduit en euros pour le client.

Classification : prédire une catégorie

Malade ou non, réadmission à 30 jours, no-show. Le modèle produit une probabilité ; la décision dépend d'un seuil, choisi d'après le coût des erreurs (page 7).

Logistic regression : une droite | **KNN (k = 15) : une courbe souple** | **Decision tree (profondeur 4) : des marches**



Mêmes données, trois decision boundaries. Bleu et rouge : les deux classes ; trait noir : probabilité 0,5.

Logistic regression

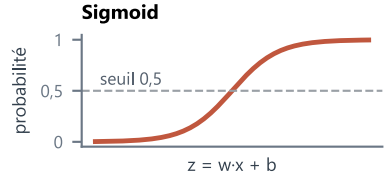
`LogisticRegression(C=1.0)`

Interprétabilité ●●●●○ Non-linéarités ●○○○○ Vitesse ●●●●●

Scaling : requis

L'idée. Une combinaison linéaire des features passée dans une **sigmoid** donne une probabilité entre 0 et 1. Entraînée en minimisant la cross-entropy (log loss) par gradient descent.

Sigmoid



QUAND L'UTILISER

- baseline de toute classification ;
- expliquer : exp(coef) = odds ratio ;
- frontière à peu près linéaire.

LIMITES

- frontière linéaire ;
- interactions à créer à la main.

Réglages clés. C (inverse de la régularisation), class_weight.

KNN

`KNeighborsClassifier(n_neighbors=15)`

Interprétabilité ●●●○○ Non-linéarités ●●●●○ Vitesse ●●○○○

Scaling : requis

L'idée. Aucun entraînement : pour un nouveau cas, on cherche les **k cas les plus proches** du train set et on les fait voter.

QUAND L'UTILISER

- petits jeux, peu de features ;
- la ressemblance a un sens (patients similaires).

LIMITES

- lent à prédire sur de gros volumes ;
- se dégrade en grande dimension (curse of dimensionality).

Réglages clés. k : petit = overfitting, grand = underfitting ; la distance.

Piège. sans scaling, la feature aux plus grands nombres décide seule de la distance.

Decision tree

`DecisionTreeClassifier(max_depth=4)`

Interprétabilité ●●●●● Non-linéarités ●●●●○ Vitesse ●●●●●

Scaling : inutile

L'idée. Une suite de questions oui ou non (âge > 65 ?), choisies une à une pour séparer au mieux les classes. Chaque feuille donne une prédiction.

QUAND L'UTILISER

- besoin de règles lisibles ;
- features hétérogènes, pas de scaling ;
- brique de base des ensembles.

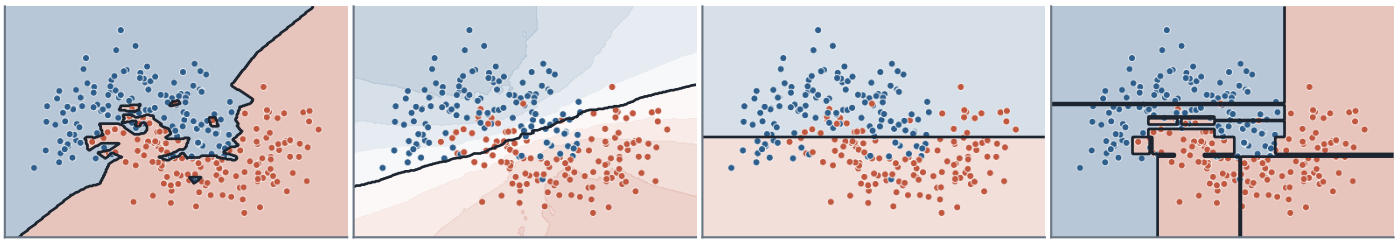
LIMITES

- instable : d'autres données, un autre arbre ;
- frontières en escalier.

Réglages clés. max_depth, min_samples_leaf.

Piège. sans limite de profondeur, il apprend le train set par cœur.

KNN, k = 1 | **KNN, k = 120** | **Arbre, profondeur 1** | **Arbre sans limite**



overfitting | underfitting | underfitting | overfitting

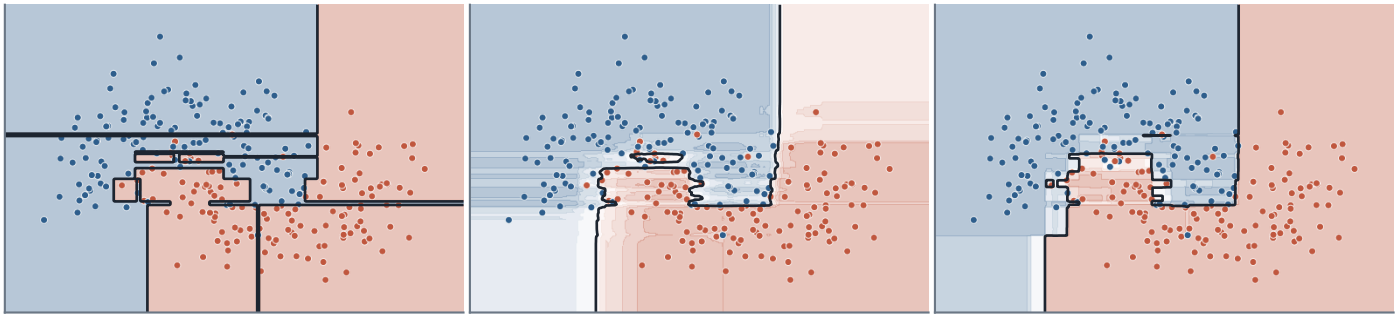
Le réglage contrôle la complexité : trop libre, le modèle suit le bruit ; trop contraint, il rate la structure.

Vu dans le cours. L'arbre sans limite de profondeur a fait nettement moins bien qu'un arbre de profondeur 5, alors que son accuracy semblait correcte : l'AUC a révélé les dégâts. Même algorithme, un seul hyperparameter.

Ensembles et interprétation

Combiner beaucoup d'arbres : sur des données tabulaires, c'est en général l'approche la plus performante. Reste à vérifier que le gain dépasse le bruit, et à expliquer le modèle.

Un arbre profond : instable | **Random forest : la moyenne lisse** | **Gradient boosting : corrections successives**



Mêmes données : un arbre seul découpe le bruit ; les ensembles donnent une frontière plus juste.

Random forest

`RandomForestClassifier(n_estimators=300)`

Données → bootstrap : B échantillons → B arbres profonds, features tirées au hasard → moyenne ou vote

Interprétabilité ●●○○○ Non-linéarités ●●●●● Vitesse ●●●○○

Scaling : inutile

L'idée. Beaucoup d'arbres profonds, chacun entraîné sur un échantillon bootstrap et sur des features tirées au hasard. Chaque arbre est instable ; leur moyenne l'est beaucoup moins, car leurs erreurs se compensent (bagging).

QUAND L'UTILISER

- très bon modèle par défaut, robuste ;
- peu de réglages ;
- score OOB (out-of-bag) gratuit.

LIMITES

- plus lourd et moins lisible qu'un arbre ;
- n'extrapole pas hors des valeurs vues.

Réglages clés. n_estimators (plus = mieux, puis plafond), max_features, min_samples_leaf.

Gradient boosting

`HistGradientBoostingClassifier()`

Arbre 1 → erreurs (résidus) → arbre 2 les corrige → ... → somme × learning rate

Interprétabilité ●●○○○ Non-linéarités ●●●●● Vitesse ●●●●○

Scaling : inutile

L'idée. Des arbres peu profonds ajoutés **un par un** : chacun corrige les erreurs des précédents. C'est du gradient descent dans l'espace des fonctions.

QUAND L'UTILISER

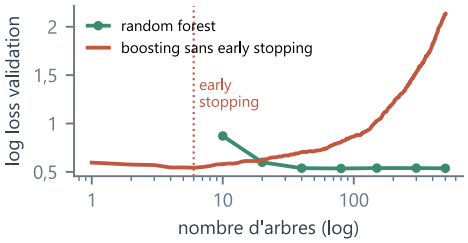
- données tabulaires : souvent le meilleur score ;
- gère manquants et gros volumes (Hist) ;
- en entreprise : XGBoost, LightGBM, CatBoost.

LIMITES

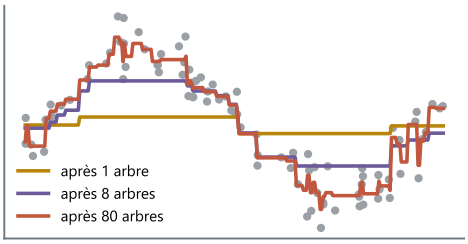
- plus de réglages ;
- overfitting si trop d'arbres sans early stopping.

Réglages clés. learning_rate et nombre d'arbres (liés), max_depth, **early_stopping**.

Plus d'arbres : plafond ou dégradation



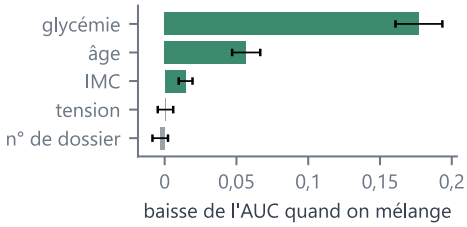
Chaque arbre corrige le précédent



	Bagging (RF)	Boosting
Arbres	profonds, en parallèle	courts, en séquence
Réduit surtout	la variance	le biais
Plus d'arbres	plafonne	peut dégrader
Réglage critique	peu	learning rate, early stopping

Interpréter un modèle complexe

Permutation importance



- **Permutation importance** : on mélange une colonne et on mesure la baisse du score. Une feature inutile (ici le n° de dossier) tombe à zéro.
- Elle mesure la **dépendance du modèle**, pas une cause. Deux features corrélées se partagent l'importance.
- **Partial dependence (PDP)** : effet moyen d'une feature sur la prédiction quand on la fait varier.
- **Analyse d'erreurs par sous-groupe** (âge, sexe, site) : une moyenne globale peut cacher un groupe mal servi. Retirer une variable sensible ne suffit pas, d'autres en sont des proxies.

Vu dans le cours. Random forest a plafonné quand on ajoutait des arbres, le boosting s'est dégradé : même geste, conséquences opposées. Et sur la régression de prix, aucun ensemble n'a battu Ridge de façon significative.

Évaluer : métriques et seuil

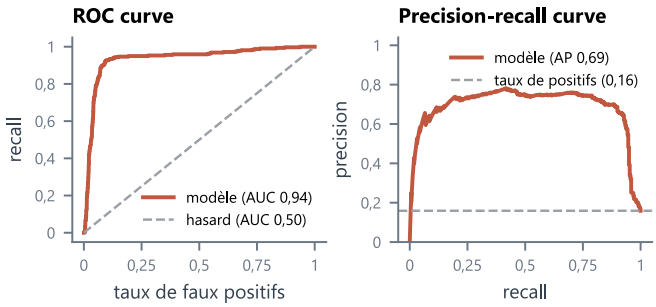
La métrique traduit le coût des erreurs dans un nombre. Elle se choisit avant d'entraîner, à partir de la décision que le modèle doit éclairer.

La matrice de confusion

	Réalité : malade	Réalité : sain
Prédit : malade	**TP** vrai positif, malade détecté	**FP** faux positif, fausse alerte
Prédit : sain	**FN** faux négatif, malade manqué	**TN** vrai négatif, sain rassuré

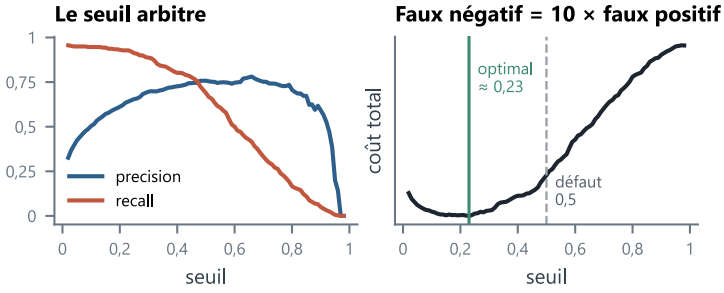
Métrique	Formule	Question posée
Accuracy	(TP + TN) / total	quelle part de bonnes réponses ?
Precision	TP / (TP + FP)	quand je dis « malade », ai-je raison ?
Recall (sensibilité)	TP / (TP + FN)	parmi les malades, combien j'en trouve ?
Specificity	TN / (TN + FP)	parmi les sains, combien je rassure ?
F1	2·P·R / (P + R)	compromis precision et recall

Courbes : classer sans choisir de seuil



- **ROC AUC** : probabilité qu'un malade reçoive un score plus haut qu'un sain. 0,5 = hasard, 1 = parfait.
- **PR AUC** (average precision) : son plancher est le taux de positifs, pas 0. Plus parlante quand les positifs sont rares.
- Une AUC élevée ne garantit pas un modèle utile **à un seuil donné**.

Le seuil se choisit, il ne se subit pas



- 0,5 n'est qu'un défaut. Baisser le seuil augmente le recall et diminue la precision.
- Écrire le coût de chaque erreur (ici un malade manqué coûte 10 fois une fausse alerte), puis prendre le seuil qui minimise le coût total, sur la validation.
- Le seuil fait partie du modèle livré : on le documente.

Quelle métrique pour quel besoin ?

Besoin	Métrique
Ne pas rater de malades	recall (sensibilité)
Éviter les fausses alertes coûteuses	precision
Classer les patients par risque	ROC AUC
Positifs rares	PR AUC, F1
Probabilités fiables	log loss, calibration
Erreur lisible en unités (régression)	MAE
Grosses erreurs très coûteuses	RMSE

Trois pièges

- **Accuracy et classes déséquilibrées** : avec 95 % de sains, prédire toujours « sain » donne 95 % d'accuracy et zéro malade trouvé.
- **La baseline peut tromper** : quand la classe positive est majoritaire, un DummyClassifier qui dit toujours « oui » obtient un F1 élevé. Toujours l'afficher à côté du modèle.
- **Choisir la métrique après coup**, parce qu'elle flatte le modèle. Elle se fixe avant d'entraîner, à partir de la décision à prendre.

Exemple chiffré : un dépistage sur 1 000 patients

1 000 patients, dont 100 malades. Le modèle en détecte 80 (TP), en manque 20 (FN) et lance 90 fausses alertes (FP) ; 810 sains sont rassurés (TN).

89 % accuracy : semble excellent

47 % precision : plus d'une alerte sur deux est fausse

80 % recall : un malade sur cinq est manqué

90 % specificity

0,59 F1

Même modèle, quatre lectures : la bonne métrique dépend de ce que coûte une fausse alerte face à un malade manqué.

Valider et régler

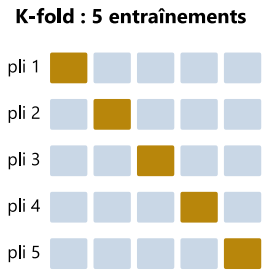
La validation dit combien un modèle vaudra sur des données nouvelles, et avec quelle incertitude. C'est elle qui choisit les hyperparameters et tranche entre deux modèles.

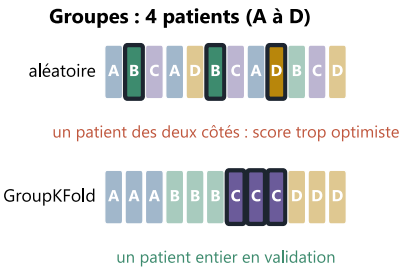
Paramètres et hyperparameters

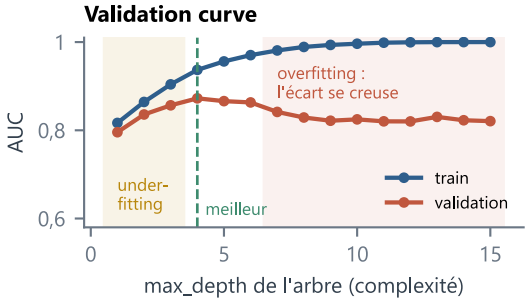
- **Paramètres** : appris par fit (coefficients, seuils des arbres, poids d'un réseau).
- **Hyperparameters** : choisis par vous avant fit (alpha, k, max_depth, learning rate). On les règle sur la validation, jamais sur le test set.

Lire une validation curve

- train bas, validation basse : **underfitting**. Modèle plus riche, meilleures features.
- train haut, validation nettement plus basse : **overfitting**. Régulariser, simplifier, plus de données.
- on choisit la complexité au sommet de la validation.







Choisir son splitter

Splitter	Principe	Quand
KFold	k entraînements, chaque ligne sert une fois en validation	lignes indépendantes
StratifiedKFold	garde la proportion de classes dans chaque fold	classification, surtout déséquilibrée
GroupKFold	toutes les lignes d'un groupe dans le même fold	plusieurs lignes par patient, hôpital, hôte
TimeSeriesSplit	entraîner sur le passé, valider sur le futur	données datées, prévision

Régler sans tricher

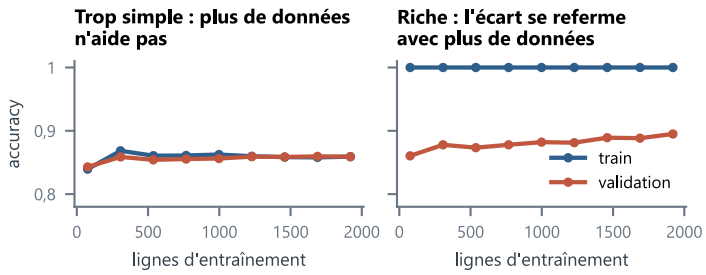
- **GridSearchCV** : toutes les combinaisons d'une grille. **RandomizedSearchCV** : un tirage, plus efficace quand il y a beaucoup d'hyperparameters.
- On met le **Pipeline entier** dans la recherche : le preprocessing est réappris dans chaque fold.
- Le meilleur score de la recherche est optimiste (on l'a choisi). Pour une estimation honnête : **nested cross-validation**, ou le test set scellé.

```
grille = {"model__max_depth": [2, 3, 5, 8]}
search = GridSearchCV(pipe, grille, cv=GroupKFold(5),
                      scoring="roc_auc")
search.fit(X_train, y_train, groups=patient_id)
```

Comparer deux modèles honnêtement

- Rapporter **moyenne ± écart-type** des folds, pas un seul chiffre.
- Utiliser les **mêmes folds** pour les deux modèles et comparer fold par fold (comparaison appariée).
- Si la différence change de signe d'un fold à l'autre, elle est **dans le bruit** : on garde le modèle le plus simple.
- Un seul split train/test est une loterie : un autre tirage donne un autre score.

Plus de données aiderait-il ? La learning curve



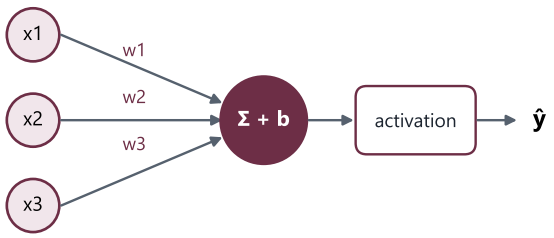
- On entraîne sur des parts croissantes du train set et on trace train et validation.
- Les deux courbes se rejoignent **bas** : le modèle est trop simple, plus de données n'y changera rien. Il faut un modèle plus riche ou de meilleures features.
- Un **écart** qui se referme lentement : plus de données aidera, tout comme la régularisation.

Vu dans le cours. Le KFold aléatoire donnait un score plus haut que le GroupKFold : il laissait les annonces d'un même hôte des deux côtés. Le plus haut n'était pas le plus juste. Et deux modèles à quelques centièmes d'écart, avec des folds qui se contredisent, sont à égalité.

Du neurone au réseau

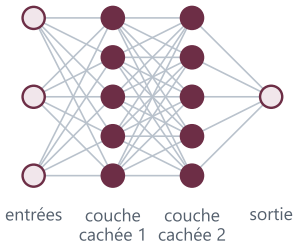
Un réseau de neurones n'est pas une rupture : c'est la logistic regression, empilée en couches avec des activations non linéaires, et entraînée par gradient descent.

Le neurone : une régression logistique



Somme pondérée des entrées plus un biais, puis une **activation**. Avec une sigmoid en sortie, un neurone seul **est** une logistic regression.

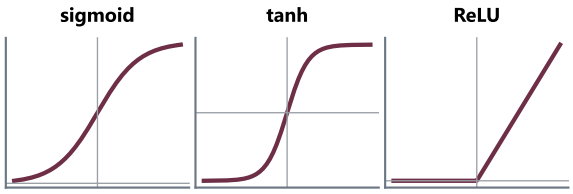
Le réseau (MLP) : empiler des couches



Chaque couche cachée fabrique de nouvelles features à partir des précédentes : le réseau apprend sa propre représentation.

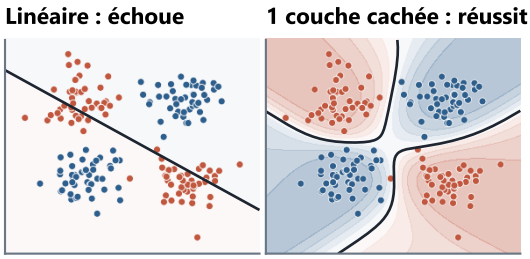
Largeur = neurones par couche ; **profondeur** = nombre de couches.

Pourquoi une activation non linéaire ?



- Sans elle, empiler des couches redonne un **modèle linéaire** : la profondeur ne sert à rien.
- **ReLU** par défaut dans les couches cachées ; **sigmoid** en sortie pour une probabilité.

Le XOR : la limite du linéaire

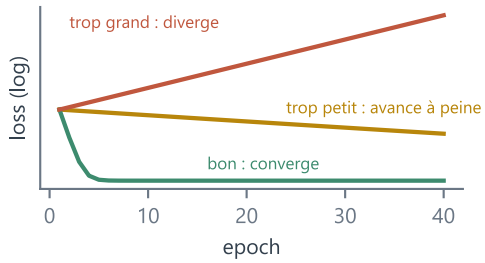


Aucune droite ne sépare ces deux classes ; une seule couche cachée suffit. C'est la raison d'être des réseaux.

Apprendre : la boucle d'entraînement

1. **Forward pass** — calculer la prédiction, couche par couche
2. **Loss** — mesurer l'erreur : MSE (régression), cross-entropy (classification)
3. **Backpropagation** — chain rule : le gradient de la loss pour chaque poids
4. **Mise à jour** — w ← w - learning rate × gradient

Le learning rate décide



Mot	Sens
epoch	un passage complet sur le train set
batch	les lignes utilisées pour un pas de gradient
iteration	un pas de mise à jour (un batch)
learning rate	la taille du pas : le réglage le plus important
gradient descent	descendre la loss dans le sens opposé au gradient

Sortie et loss selon la tâche

Tâche	Dernière couche	Loss PyTorch
Régression	1 neurone, sans activation	nn.MSELoss()
Classification binaire	1 neurone (un logit)	nn.BCEWithLogitsLoss(), sigmoid incluse
Plusieurs classes	k neurones (k logits)	nn.CrossEntropyLoss(), softmax incluse

La loss « with logits » applique elle-même la sigmoid : ne pas en ajouter une seconde en sortie du modèle.

Quatre idées reçues

- **« Plus de couches, c'est mieux. »** Sur des données tabulaires modestes, un réseau profond peut faire moins bien qu'une seule couche.
- **« La backpropagation est un algorithme à part. »** C'est la chain rule, appliquée de la sortie vers l'entrée.
- **« L'activation est un détail. »** Sans elle, le réseau redevient linéaire.
- **« Un réseau peut tout apprendre. »** Qu'une solution existe (universal approximation) ne garantit pas qu'on la trouve avec des données finies et bruitées.

Vu dans le cours. Le gradient calculé à la main sur un petit réseau 2-2-1 correspondait exactement à celui de PyTorch (autograd) : la backpropagation n'est que la chain rule. Et ce sont la même loss et le même gradient descent qu'en régression et en classification.

Entraîner et généraliser

Un réseau qui entraîne n'est pas un réseau qui généralise. On lit les courbes de loss dès la première epoch, et on régularise une chose à la fois.

La boucle PyTorch, ligne par ligne

```
model = nn.Sequential(nn.Linear(d, 64), nn.ReLU(),
                      nn.Linear(64, 1))
loss_fn = nn.BCEWithLogitsLoss()   # sigmoid incluse
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(n_epochs):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()              # 1. remettre à zéro
        loss = loss_fn(model(xb), yb) # 2. forward + loss
        loss.backward()              # 3. backpropagation
        opt.step()                   # 4. mise à jour
    model.eval()                     # 5. évaluer
    with torch.no_grad(): ...        #    sans gradient
```

Oublier zero_grad accumule les gradients ; oublier model.eval() laisse dropout et batch norm en mode entraînement pendant l'évaluation.

Les optimizers

Optimizer	Principe	Usage
SGD	un pas proportionnel au gradient	simple, très sensible au learning rate
SGD + momentum	garde une inertie des pas précédents	descente plus lisse et plus rapide
Adam	un learning rate adapté à chaque poids	bon défaut : lr = 1e-3

- **Batch** petit : gradient bruité, parfois meilleure généralisation ; grand : plus rapide par epoch, learning rate à ajuster.
- **Standardiser** les entrées : sinon la loss stagne ou explose.
- Tracer train **et** validation dès la première epoch.

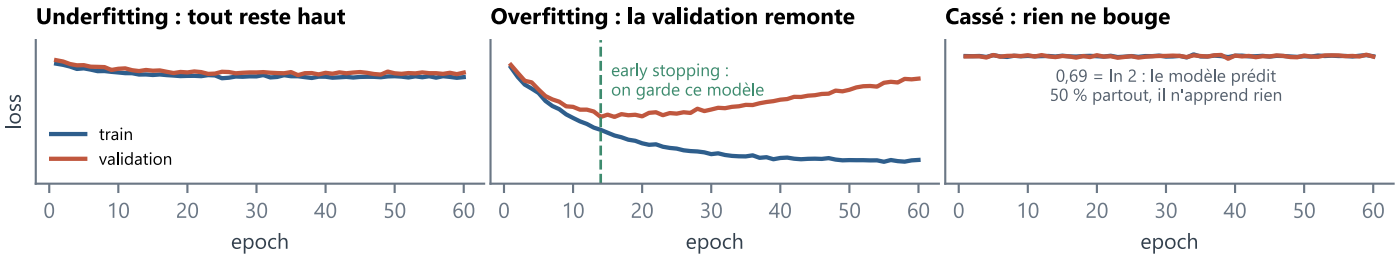


Schéma : les trois profils de courbes de loss à reconnaître.

Underfitting

- plus de capacité (neurones, couches) ;
- entraîner plus longtemps ;
- vérifier le learning rate et les features.

Overfitting

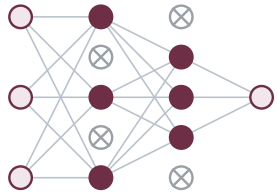
- early stopping d'abord ;
- weight decay, dropout ;
- plus de données, data augmentation.

Cassé

- learning rate trop grand (loss qui explose ou NaN) ;
- entrées non standardisées ;
- bug : labels décalés, mauvaise loss, shapes.

Régulariser un réseau

Technique	Ce qu'elle fait	Coût ou piège
Early stopping	arrête quand la validation ne s'améliore plus et garde le meilleur modèle	le moins cher : à faire en premier ; régler la patience
Weight decay (L2)	pénalise les grands poids : c'est Ridge	un hyperparameter de plus
Dropout	éteint des neurones au hasard pendant l'entraînement	n'aide pas toujours ; model.eval() indispensable
Batch norm, layer norm	normalise les activations, stabilise l'entraînement	batch norm : différente en train et en eval (layer norm, non)
Data augmentation	crée des variantes réalistes (image tournée, recadrée)	doit respecter le sens des données



Dropout : à chaque pas, des neurones éteints au hasard.

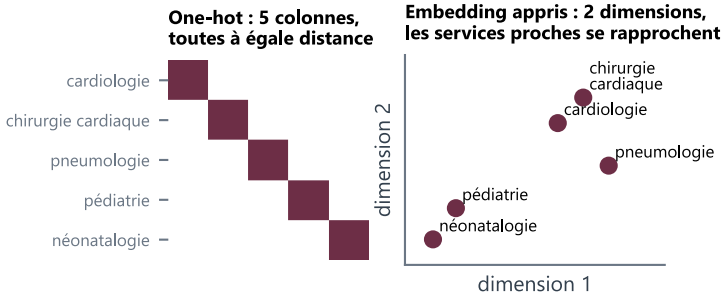
Ablation : changer un seul facteur à la fois, sur plusieurs seeds, et comparer l'écart au bruit entre seeds. Les régularisations interagissent : les empiler toutes n'est pas une stratégie.

Vu dans le cours. Le dropout n'a pas systématiquement aidé. Et sur nos données tabulaires, le réseau n'a pas battu le gradient boosting : le deep learning n'est pas un choix par défaut.

Représentations et texte

La vraie force du deep learning : apprendre des représentations (embeddings) pour des données que les tableaux décrivent mal, comme le texte et les images.

One-hot ou embedding ?



- **One-hot** : une colonne par catégorie, toutes à la même distance. Rien ne dit que deux services se ressemblent.
- **Entity embedding** (nn.Embedding) : chaque catégorie devient un petit vecteur **appris** ; les catégories qui se comportent pareil pour la tâche se rapprochent.
- Utile quand une variable a des centaines de modalités (codes, médicaments, communes). Illustration schématique.

Texte : TF-IDF d'abord

`TfidfVectorizer() + LogisticRegression()`

Interprétabilité ●●●●○ Non-linéarités ●●○○○ Vitesse ●●●●●

Scaling : inutile

L'idée. Chaque document devient un vecteur de mots pondérés : fréquent dans ce texte, rare ailleurs = important. Un modèle linéaire fait le reste.

QUAND L'UTILISER

- toujours, comme baseline texte ;
- peu de données ;
- besoin de voir les mots qui pèsent.

LIMITES

- ignore l'ordre et le sens des mots ;
- synonymes vus comme différents.

Réglages clés. ngram_range, min_df, et C du modèle.

Encoder pré-entraîné

`SentenceTransformer(...).encode(textes)`

Interprétabilité ●○○○○ Non-linéarités ●●●●● Vitesse ●●○○○

Scaling : selon l'encoder

L'idée. Un modèle entraîné sur des millions de textes transforme une phrase en vecteur de sens. **Gelé** (frozen), il s'utilise comme une simple étape de preprocessing : c'est le transfer learning.

QUAND L'UTILISER

- peu de données étiquetées ;
- le sens compte plus que les mots exacts.

LIMITES

- plus lent et plus lourd ;
- vecteurs difficiles à interpréter ;
- vocabulaire médical parfois mal couvert.

Réglages clés. le choix de l'encoder ; geler ou affiner (fine-tuning).

Fusion multimodale : tabulaire + texte

Tabulaire : âge, constantes → standardisation

Texte : compte rendu → encoder pré-entraîné gelé → vecteur de sens

concaténation → MLP → prédiction

- Ce que le gradient boosting ne fait pas nativement : combiner des types de données différents dans un même modèle.
- Pièges : échelles différentes entre les branches, une branche qui écrase l'autre.
- Toujours mesurer la **valeur marginale** du texte : le modèle complet contre le tabulaire seul, sur les mêmes folds.

Les grandes architectures, pour s'orienter

Architecture	Pour quelles données
MLP	tabulaire, vecteurs d'embeddings
CNN	images, signaux : des filtres détectent des motifs locaux
RNN, Transformer	séquences et texte ; l'attention relie les mots entre eux
Modèle pré-entraîné	réutiliser ce qu'un grand modèle a appris ailleurs

Deep learning ou gradient boosting ?

Vos données	Premier choix	Pourquoi
Tabulaires	gradient boosting	plus précis, plus rapide, moins de réglages
Texte, images, son	deep learning pré-entraîné	les représentations apprises dominent
Tabulaire + texte	fusion, ou boosting sur embeddings	mesurer l'apport du texte
Peu de données	modèles classiques, transfer learning	un réseau entraîné de zéro sur-apprend

Vu dans le cours. Sur le texte, la représentation apprise a apporté de l'information ; sur la partie tabulaire, le boosting restait meilleur. Le deep learning gagne là où il apporte une représentation que l'on n'aurait pas su construire.

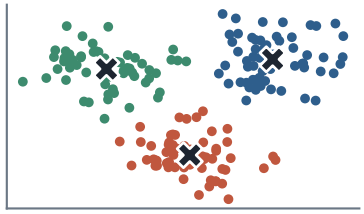
Non supervisé : sans target

On ne prédit plus une réponse connue : on cherche une structure. Des groupes, des directions principales, des cas inhabituels. Le jugement humain fait partie de la méthode.

k-means

`KMeans(n_clusters=3, n_init=10)`

k-means (k = 3) : des centres



L'idée. Place k centres, affecte chaque point au plus proche, déplace chaque centre à la moyenne de ses points, et recommence jusqu'à stabilité.

QUAND L'UTILISER

- groupes compacts, à peu près ronds ;
- gros volumes ; premier essai.

LIMITES

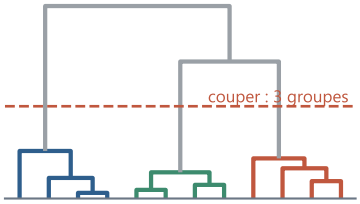
- k à choisir ;
- formes allongées ou imbriquées ratées ;
- tout point reçoit un cluster.

Réglages clés. k (elbow method, silhouette), n_init.

Clustering hiérarchique

`AgglomerativeClustering(linkage="ward")`

Hiérarchie : le dendrogramme



L'idée. Fusionne pas à pas les groupes les plus proches. Le **dendrogramme** garde l'histoire ; on choisit le nombre de groupes en coupant l'arbre.

QUAND L'UTILISER

- voir la structure à toutes les échelles ;
- petits et moyens jeux.

LIMITES

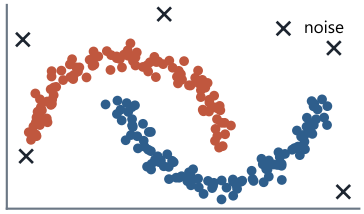
- lent sur de gros volumes ;
- une fusion n'est jamais défaite.

Réglages clés. linkage (ward, average), hauteur de coupe.

DBSCAN

`DBSCAN(eps=0.3, min_samples=5)`

DBSCAN : densité et noise



L'idée. Un cluster est une zone **dense** ; les points isolés deviennent du **noise**. Pas de k à choisir.

QUAND L'UTILISER

- formes quelconques ;
- repérer du bruit, des cas isolés.

LIMITES

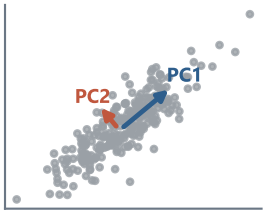
- eps délicat à régler ;
- densités très différentes : problème.

Réglages clés. eps, min_samples.

PCA

`PCA(n_components=0.90)`

PCA : PC1 garde 93 %



L'idée. Trouve les directions où les données varient le plus (PC1, PC2…) et projette dessus : beaucoup de features résumées en quelques composantes.

QUAND L'UTILISER

- visualiser en 2D ;
- features très redondantes ;
- compresser.

LIMITES

- ne voit jamais la target ;
- linéaire ;
- composantes moins lisibles (lire les loadings).

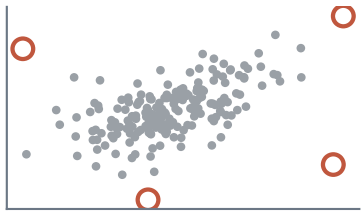
Réglages clés. nombre de composantes (explained variance, souvent 90 %).

Piège. standardiser avant, sinon la feature aux plus grands nombres devient PC1.

Anomaly detection

`IsolationForest()`

Anomalies : loin de tout



L'idée. Apprendre à quoi ressemble le normal et signaler ce qui s'en écarte : point loin de tout centre, noise de DBSCAN, ou méthode dédiée (Isolation Forest, Local Outlier Factor).

QUAND L'UTILISER

- fraude, panne de capteur ;
- erreur de saisie, cas clinique atypique.

LIMITES

- le « normal » doit être représentatif ;
- une anomalie est à examiner, pas à supprimer d'office.

Réglages clés. contamination (part attendue d'anomalies), seuil d'alerte.

Évaluer sans target

- **Silhouette** (de -1 à 1) : chaque point est-il plus proche de son groupe que du groupe voisin ?
- **Stabilité** : autre graine, autre échantillon, k voisin : les groupes restent-ils ?
- **Interprétation** : décrire chaque groupe ; un groupe qu'on ne sait pas décrire ne sert à rien.
- Si des labels existent : **ARI** pour comparer, jamais pour entraîner.

Piège. L'algorithme renvoie toujours une réponse : demandez 4 clusters à du bruit pur, vous en aurez 4. Le scaling et le choix des features décident de ce que « proche » veut dire.

SYNTHÈSE

Synthèse, pièges et checklist

Une page à relire avant chaque projet : quel algorithme pour quoi, les erreurs les plus fréquentes, et ce qu'il faut vérifier avant de livrer.

Tous les algorithmes en un coup d'œil

Algorithme	Tâche	Idée en une ligne	Scaling	Lisibilité /5	À choisir quand
DummyRegressor, DummyClassifier	les deux	prédire la moyenne ou la classe majoritaire	non	5	toujours, comme plancher
Linear regression	régression	droite des moindres carrés	non	5	relation linéaire, besoin d'expliquer
Ridge, Lasso	régression	linéaire + pénalité	oui	4	beaucoup de features corrélées
Logistic regression	classification	linéaire + sigmoid	oui	4	baseline, odds ratios à expliquer
KNN	les deux	vote des k plus proches voisins	oui	3	petit jeu, ressemblance pertinente
Decision tree	les deux	questions oui ou non successives	non	5	règles lisibles, profondeur limitée
Random forest	les deux	moyenne d'arbres décorrélés	non	2	tabulaire, modèle robuste par défaut
Gradient boosting	les deux	arbres qui corrigent les précédents	non	2	tabulaire, meilleur score visé
MLP (réseau)	les deux	couches + activations, gradient descent	oui	1	beaucoup de données, représentations
TF-IDF, embeddings	texte	mots pondérés, vecteurs de sens	non	3 à 1	comptes rendus, notes libres
k-means, DBSCAN, hiérarchique	non supervisé	regrouper sans target	oui	4	segmenter, explorer
PCA	non supervisé	directions de variance maximale	oui	3	visualiser, compresser

Dix pièges à éviter

1. Pas de baseline : impossible de dire si le modèle apprend quelque chose.
2. Explorer ou prétraiter avant le split : le test set n'est plus neutre.
3. Une feature qui n'existera pas au moment de prédire (data leakage).
4. Le même patient en train et en test : score trop optimiste.
5. Juger sur l'accuracy avec des classes déséquilibrées.
6. Garder le seuil 0,5 sans réfléchir au coût des erreurs.
7. Conclure « A est meilleur que B » sur un écart plus petit que le bruit des folds.
8. Régler les hyperparameters sur le test set.
9. Lire une importance ou un coefficient comme une cause.
10. Choisir le deep learning par réflexe sur des données tabulaires.

Checklist avant de livrer un modèle

- ☐ La baseline est battue, nettement.
- ☐ Split et cross-validation respectent les groupes et le temps.
- ☐ Aucune feature indisponible au moment de prédire.
- ☐ Métrique et seuil justifiés par le coût des erreurs.
- ☐ Score rapporté avec son incertitude (moyenne ± écart-type).
- ☐ Performance vérifiée par sous-groupe (âge, sexe, site).
- ☐ Test set utilisé une seule fois, à la fin.
- ☐ Usage prévu, limites et dates des données écrits (model card).

Le vocabulaire en un coup d'œil

Target la colonne à prédire (y).

Feature une variable d'entrée, une colonne de X.

Train, validation, test set apprendre, choisir, puis mesurer une fois à la fin.

Baseline modèle naïf qui fixe le plancher à battre.

Overfitting le modèle apprend le bruit du train set et généralise mal.

Underfitting modèle trop simple : il rate la structure.

Bias-variance trop rigide (biais) ou trop sensible aux données (variance).

Data leakage une information indisponible au moment de prédire s'infiltre.

Pipeline preprocessing et modèle enchaînés, appris sur le train set.

Cross-validation plusieurs découpages train et validation, pour un score stable.

Hyperparameter réglage choisi avant l'entraînement : k, alpha, max_depth.

Regularization pénalité qui simplifie le modèle : Ridge, Lasso, weight decay.

Decision boundary frontière où la prédiction change de classe.

Seuil (threshold) probabilité à partir de laquelle on prédit « positif ».

Loss la mesure d'erreur que l'entraînement fait baisser.

Gradient descent avancer à petits pas dans le sens qui fait baisser la loss.

Learning rate la taille de ces pas.

Epoch un passage complet sur le train set.

Embedding vecteur appris qui représente une catégorie, un mot, une phrase.

Clustering regrouper des observations sans target.

Ensemble plusieurs modèles combinés : bagging, boosting.

Un bon modèle n'est pas celui qui a le meilleur score : c'est celui dont on peut **défendre** le score, expliquer les erreurs et assumer les limites.